Práctica 1: Plants versus Zombies

Miriam García Soto

Departamento de Sistemas Informáticos y Computación



Curso 2026/2027

Contenidos

1. Arquitectura General
2. Vista y Controlador
3. Modelo y Objetos
4. Secuencia y Normas

1 Arquitectura General

2 Vista y Controlador

3 Modelo y Objetos

4 Secuencia y Normas

Arquitectura general de la plantilla

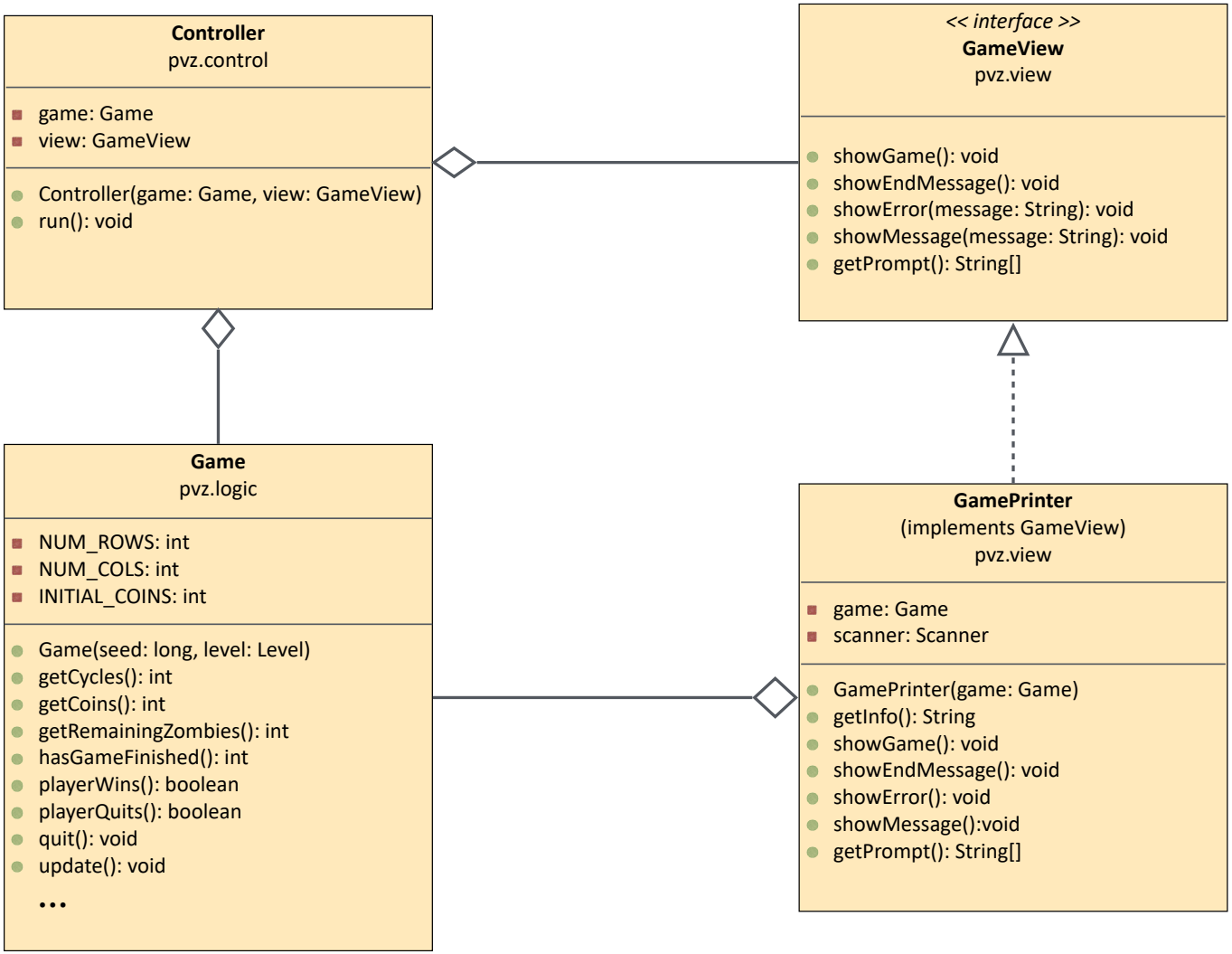
Modelo-Vista-Controlador

- La **Vista** se corresponde con el paquete `view`. Ahí se encuentran las clases encargadas de mostrar el juego, en concreto la clase GamePrinter.
- El **Controlador** se corresponde con el paquete `control`. Ahí se encuentra la clase `Controller`, encargada de gestionar el "ciclo de vida" del juego.
- El **Modelo** se corresponde con el paquete `logic`, donde se encuentra la lógica del juego, es decir, las reglas del juego y el manejo de todos los objetos del juego.

Puesta en marcha en Main

```
Game game = new Game(seed, level);
Controller controller = new Controller(game);
controller.run();
```

Puesta en marcha: Visión de componentes



1 Arquitectura General

2 Vista y Controlador

3 Modelo y Objetos

4 Secuencia y Normas

Vista: Métodos de GameView

```java
// show methods
public void showGame();
public void showEndMessage();
public void showError(String message);
public void showMessage(String message);

// get data from view methods
public String getPrompt();
```

Controlador: clase `Controller`

Implementa el método `run()`, invocado desde `PlantsVsZombies`, que ejecuta una partida:

```
public void run() {
        view.showGame();
        while (!game.hasGameFinished()) {
                // TODO fill your code
                // Prompt the user for a line (getPrompt)
                // Execute the user's command (method)
        }
        view.showEndMessage();
}
```

1 Arquitectura General

2 Vista y Controlador

3 Modelo y Objetos

4 Secuencia y Normas

Modelo: clase `Game`

- La clase `Game` encapsula la lógica del juego.
- Habrá una **única instancia** de `Game` en el programa.
- Contiene una instancia de `SunflowerList`, `PeashooterList` y `ZombiesManager`.
- También mantiene el contador de ciclos y monedas, entre otros valores.

Arrays de plantas: `SunflowerList`, `PeashooterList`

Clases que almacenan y gestionan las plantas presentes en el juego.
Por ejemplo, `SunflowerList`:

```
private int numberOfSunflowers;
private final Sunflower[] sunflowers;
...
numberOfSunflowers = 0;
sunflowers = new Sunflower[100];
```

- Tiene métodos para:
  - Manipular esas estructuras de datos (p.ej., para añadir una nueva planta).
  - Redirigir a los objetos peticiones provenientes de `Game` (p. ej., update)

Clase ZombiesManager

- **Atributos:** El juego (`Game`), el nivel, el número de zombies que quedan por aparecer, el array de zombies y el objeto Ramdon
- **Algunos métodos:**
  - public void update(): actualiza la lista de zombies.
  - public String iconInPosition(Position p): devuelve el icono de zombie según su posición. Se lo pedirá a la lista de zombies.
  - public boolean addZombie(): situa un zombie en una fila aleatoria.
  - public boolean addZombie(int row): situa un zombie en una fila dada.

1 Arquitectura General

2 Vista y Controlador

3 Modelo y Objetos

4 Normas

Recordatorios importantes

- **Cualquier rotura de la encapsulación será motivo de suspenso.**
- Ejemplos de rotura de la encapsulación:
  - Atributos públicos.
  - Métodos *getters* públicos que devuelven objetos.
  - Métodos públicos que devuelven listas o arrays.
- `Game` no debería tener ningún bucle.
- No se debe añadir ningún `System.out...`, ni en `Controller` ni en `Game`, ni en ninguna otra clase (todos están ya en la vista).